

Hash Table / dict / set

Review Sheet

A hash table turns "have I seen this before / where is it" into a one-shot $O(1)$ lookup — the single highest-frequency trick across every problem set. This book nails down the concept, gives you five core templates, walks through six classic problems, and smooths the transition from Java-brain to Python.

- Python 3
- Tutor style
- Includes diagrams
- Includes FAQ
- Java→Python

00 Review Order

Start with the simplest — "does it exist" — then progressively layer on counting, position-tracking, and grouping, and finish with a combined problem. Each box is the foundation for the next.

01 ← start

set — dedup

Seen before?

02

dict — counting

How many times?

03

dict — index

Where is it?

04

dict — grouping

Sort by trait

05

set — intersection

Shared elements

06

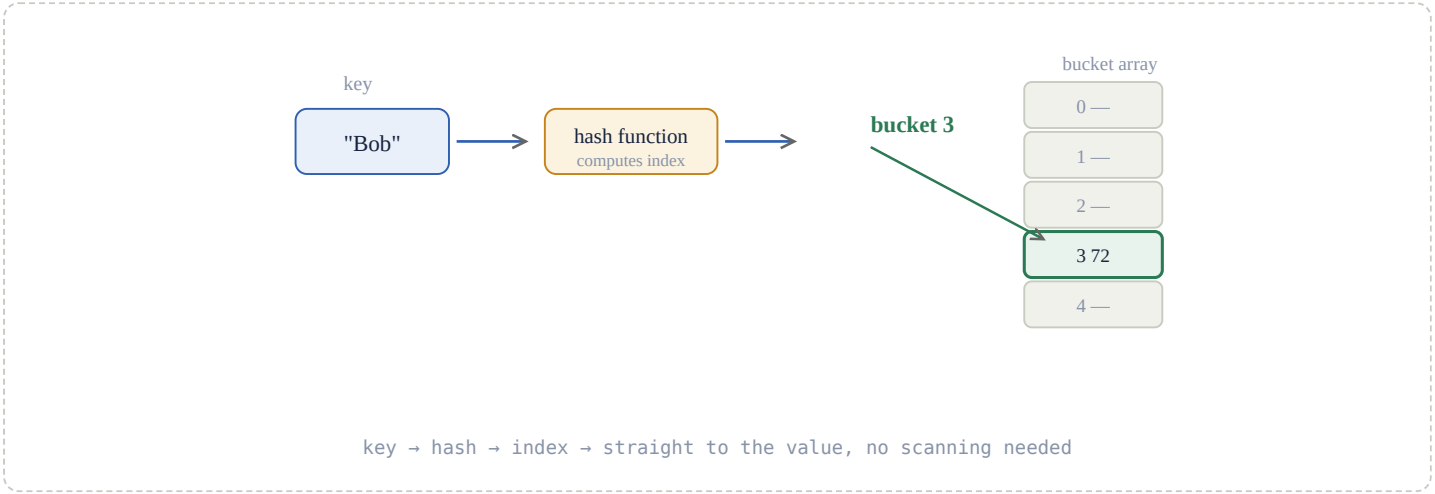
Combined

Mixing it up

dict or set — one-sentence rule: if you need to **keep a value alongside** the key (a count, an index, the original word), use `dict` ; if you only care **whether it exists**, use `set` . A set is really just "a dict with only keys, no values."

01 Hash Table Concept

A plain list search is worst-case $O(n)$ — scan from front to back. A hash table uses a **hash function** to turn a key directly into a "bucket index," jumping straight there in one step — average $O(1)$.



dict vs set

	WHAT IT STORES	TYPICAL USE
dict	key → value pairs	Counting, recording position, grouping, building a mapping
set	Only keys (no values)	Deduplication, checking existence, finding duplicates, intersections

02 Five Core Templates

Almost every hash-table problem is a variation on one of these five shapes.

① dict – counting

count

```
counts = {}
for item in data:
    if item not in counts:      # first time seeing this key, initialize to 0
        counts[item] = 0
    counts[item] += 1

# Pythonic: counts[item] = counts.get(item, 0) + 1
# Even shorter: from collections import Counter; Counter(data)
```

② set – dedup / existence check

dedup

```
seen = set()
for item in data:
    if item in seen:           # seen it before → duplicate found
        return True
    seen.add(item)             # add it, keep growing seen
return False
```

③ dict – index → position

index map

```
index_map = {}
for index, value in enumerate(numbers):
    index_map[value] = index    # O(1) "where is this value" lookup
```

④ dict – key → list, grouping

group

```
groups = {}
for item in data:
    key = get_key(item)        # extract the "shared trait"
    if key not in groups:      # first time this key appears, make a list
        groups[key] = []
    groups[key].append(item)
return list(groups.values())

# Alternative: from collections import defaultdict; groups = defaultdict(list)
```

⑤ set – intersection

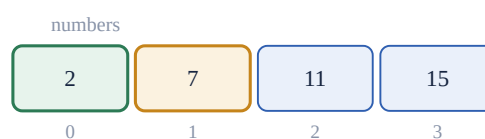
intersect

```
set1 = set(nums1)
result = set()
for number in nums2:
    if number in set1:        # exists in both → keep it
        result.add(number)    # result is a set, dedup is automatic
return list(result)

# Shortcut: list(set(nums1) & set(nums2))
```

03 Two Sum

Given `numbers` and `target`, find two indices whose values sum to target. Uses **Template ③ (record index)**.



reach 7 (index 1): needed = 9 - 7 = 2

2 is already in seen (index 0) → return [0, 1]

target = 9 · walk once, ask each

step "have I seen my complement?"

needed = target - current number

```
def two_sum(numbers, target):
    seen = {}                                # value → index
    for index, number in enumerate(numbers):
        needed = target - number             # what I still need
        if needed in seen:                  # already seen it → match found
            return [seen[needed], index]
        seen[number] = index                # no match yet, record myself for later
    return []                               # no answer found
```

Core idea: don't use two nested loops to try every pair (that's $O(n^2)$). Instead, walk through once — at each number, ask "have I seen **target minus me** before?" A hash table turns that "have I seen it, and where" into $O(1)$, bringing the whole thing down to $O(n)$.

Why check before storing? Check **needed** first, *then* store yourself — this naturally avoids using the same element twice.

04 Contains Duplicate

Determine whether a list has any duplicate numbers. The textbook application of **Template ② (set dedup)**.

contains_duplicate

dedup

```
def contains_duplicate(numbers):
    seen = set()
    for number in numbers:
        if number in seen:                # seen it again → duplicate exists
            return True
        seen.add(number)
    return False                          # made it through with no repeats → no duplicates
```

Q Why **set** here instead of **dict** ?

Because you only care whether something has **appeared before** — you don't need to track a count or a position. That's the textbook **set** scenario. An empty list naturally returns **False** (the loop never runs even once).

Empty list → False

All unique → False

First and last equal → still counts as a duplicate

05 First Unique Character

Return the first character that appears exactly once; return None if there isn't one. **Template ① (counting) + two passes**.

first_unique_char – count once, then re-scan to find

count

```
def first_unique_char(text):
    counts = {}
    for char in text:                      # pass 1: tally every character's count
        if char not in counts:
            counts[char] = 0
        counts[char] += 1

    for char in text:                      # pass 2: walk in original order, find count==1
        if counts[char] == 1:
            return char
    return None
```

Q Why does this need two passes instead of one?

The first pass builds the complete **counts** table — you can't know a character's **final** total count until you've scanned the whole string. The second pass walks **text** again **in its original order**, checking each character's count — this preserves "the first one in the original string with count 1," which a single pass over an unordered dict couldn't guarantee.

06 Is Anagram

Check whether two strings are anagrams of each other (same letters, same counts). Count each string separately, then compare the two count dicts directly.

is_anagram – build two counters, compare directly

count compare

```
def is_anagram(s, t):
    count_s = {}
    count_t = {}
    for char in s:
        count_s[char] = count_s.get(char, 0) + 1
    for char in t:
        count_t[char] = count_t.get(char, 0) + 1
    return count_s == count_t    # dict == dict compares key+value pairs directly!

# Shortcut: return Counter(s) == Counter(t)
```

Q Why not manually loop `for k in count_s: ... count_t[k] ...` to compare?

If some key `k` exists in `count_s` but not in `count_t`, then `count_t[k]` throws `KeyError`. Comparing directly with `count_s == count_t` is both simpler *and* safer — Python's dict comparison already checks that every key and value matches on both sides, with no crash risk.

Q What's the actual difference between `!=`, `is not`, and `==`?

`==` / `!=` compare **value equality** (do the contents match); `is` / `is not` compare **identity** (are they the literal same object in memory). Dict/list/set comparisons for content should always use `==` / `!=` — using `is` here is a classic Java-brain habit that doesn't transfer to Python correctly.

07 Group Anagrams

Group a list of words so that anagrams of each other end up together. **Template ④ (grouping)** — the trick is finding a key that represents "belongs to this group."



Sort a word's letters and use that as the key — anagrams fall into the same bucket naturally

group_anagrams

group

```
def group_anagrams(words):
    groups = {}
    for word in words:
        key = "".join(sorted(word))    # key move: sorted letters as the key
        if key not in groups:
            groups[key] = []
        groups[key].append(word)
    return list(groups.values())
```

When you're stuck, think this way: the universal pattern for a "grouping" problem is "find a key that represents the whole group." Anagrams share the trait "same set of letters" — sorting the letters gives you a unified identifier: `eat/tea/ate → "aet"`. Same key means they automatically land in the same bucket.

08 Intersection of Two Lists

Return the shared elements of two lists, with no duplicates. **Template ⑤ (intersection)**.

```
def intersection(nums1, nums2):
    set1 = set(nums1)          # convert one to a set: dedup + O(1) lookup
    result = set()
    for number in nums2:
        if number in set1:
            result.add(number)  # result is a set, naturally dedup'd
    return list(result)

# Shortcut: return list(set(nums1) & set(nums2))
```

The two sets each have their own job: `set1` handles "quickly checking whether an element from `nums2` is in `nums1`"; `result` handles "collecting and auto-deduping." No overlap returns `[]`. Watch out: set output order is **not guaranteed** — both `[9,4]` and `[4,9]` are valid answers.

09 Java → Python Mindset Conversion

Coming from writing Java (AP CSA) straight into Python, the easiest trap is "over-manual-izing." Python's built-in structures can do a lot with **direct comparison**, **direct checks** — no need to loop it out by hand.

SCENARIO	JAVA INSTINCT (VERBOSE)	PYTHON WAY (DIRECT)
Compare two maps	Loop keys one by one, risk <code>KeyError</code>	<code>d1 == d2</code>
Check membership	<code>containsKey</code> / <code>contains</code>	<code>k in d</code> / <code>x in s</code>
Safe value lookup	<code>containsKey</code> first, then <code>get</code>	<code>d.get(k, default)</code>
Compare values for equality	Mixing up <code>==</code> with object identity	<code>!=</code> for values, <code>is</code> for identity

Python's `==` already compares whole-structure equality

`==` semantics

```
{ "a": 1, "b": 2 } == { "b": 2, "a": 1 }  # True — dicts compare key+value, order doesn't matter
[1, 2, 3] == [3, 2, 1]                  # False — lists care about order
{1, 2, 3} == {3, 2, 1}                  # True — sets don't care about order
```

Bring this habit with you: in Python, membership checks use `in`, safe lookups use `.get()`, and whole-structure comparisons use `==` / `!=` directly. If you find yourself manually looping to check "does this key exist" or "are these two structures equal," stop — there's almost certainly a built-in, more Pythonic way to say the same thing in one line.

10 Bug Comparison Table

WRONG / VERBOSE	BETTER / CORRECT
<code>counts[c] += 1</code> (key may not exist yet)	<code>counts[c] = counts.get(c, 0) + 1</code>
<code>groups[k].append(x)</code> with no init check	Check if <code>k not in groups</code> : <code>groups[k]=[]</code> first, or use <code>setdefault</code>
Manually looping to compare two dicts key by key	<code>d1 == d2</code>
<code>a is not b</code> to compare values	<code>a != b</code>
Forgetting to actually call <code>seen.add()</code>	<code>seen.add(x)</code> every time you record a new value
Repeating "check then insert" by hand	Reach for <code>.get()</code> or <code>setdefault()</code>
Confusing <code>==/!=</code> with <code>is/is None</code>	Value comparison → <code>==/!=</code> ; identity/None check → <code>is/is None</code>

11 Exercises

Grouped by which template each one drills.

◆ COUNTING (TEMPLATE ①)

1. **Easy** Count how many times each item appears in a list, without using Counter.
2. **Medium** Find the most frequent element (handle ties for first place).

◆ SET / DEDUP (TEMPLATE ②)

1. **Easy** Implement `contains_duplicate` and verify against an empty list and an all-unique list.

◆ INDEX / POSITION (TEMPLATE ③)

1. **Medium** Implement `two_sum`, and trace through it by hand on `[2,7,11,15]`, `target=9`.

◆ GROUPING (TEMPLATE ④)

1. **Medium** Implement `group_anagrams` and verify `["eat","tea","tan","ate","nat","bat"]` groups correctly.

◆ INTERSECTION (TEMPLATE ⑤) & COMBINED

1. **Easy** Implement `intersection` and confirm it handles no-overlap correctly (returns []).
2. **Medium** Combined: given a list of words, return true if any two of them are anagrams of each other (mix Template ② and ④).